Guía básica para trabajar con funciones en Python

1. ¿Qué es una función?

Una **función** es un bloque de código que realiza una tarea específica.

Sirve para organizar mejor un programa y evitar repetir código muchas veces.

Por ejemplo, si queremos saludar a una persona, podríamos escribir:

```
print("Hola, bienvenido a Python")
```

Pero si necesitamos mostrar ese saludo muchas veces, es mejor crear una función:

```
def saludar():
    print("Hola, bienvenido a Python")
```

Luego podemos usarla cuando queramos:

```
saludar()
```

Salida:

```
Hola, bienvenido a Python
```

---

2. ¿Para qué sirven las funciones?

Las funciones sirven para:

- Reutilizar código.
- Organizar mejor un programa.
- Dividir un problema grande en partes pequeñas.
- Evitar escribir muchas veces las mismas instrucciones.
- Hacer que el código sea más fácil de leer y mantener.

Por ejemplo, en vez de repetir esto varias veces:

```
print("Bienvenido")
```

```
print("Este programa calcula promedios")
print("Ingrese sus datos")
```

Podemos crear una función:

```
def mostrar_bienvenida():
    print("Bienvenido")
    print("Este programa calcula promedios")
    print("Ingrese sus datos")
```

Y luego llamarla:

```
mostrar_bienvenida()
```

---

3. Crear una función

Para crear una función en Python usamos la palabra reservada `def`.

```
def nombre_de_la_funcion():
    instrucciones
```

Ejemplo:

```
def mostrar_mensaje():
    print("Estoy aprendiendo funciones en Python")
```

Para ejecutar la función, debemos llamarla por su nombre:

```
mostrar_mensaje()
```

Programa completo:

```
def mostrar_mensaje():
    print("Estoy aprendiendo funciones en Python")

mostrar_mensaje()
```

Salida:

```
Estoy aprendiendo funciones en Python
```

---

4. Importancia de la indentación

En Python, la indentación indica qué instrucciones pertenecen a la función.

Correcto:

```
def saludar():
    print("Hola")
    print("Bienvenido")
```

Incorrecto:

```
def saludar():
print("Hola")
print("Bienvenido")
```

El segundo ejemplo genera error porque las instrucciones no están indentadas.

La indentación normalmente se hace con 4 espacios.

---

5. Llamar a una función

Crear una función no significa que se ejecute automáticamente.

Primero se define:

```
def saludar():
    print("Hola")
```

Luego se llama:

```
saludar()
```

Programa completo:

```
def saludar():
    print("Hola")
```

saludar()
saludar()
saludar()

Salida:

Hola
Hola
Hola

Una función puede llamarse muchas veces.

---

6. Funciones con parámetros

Un **parámetro** es un dato que la función necesita recibir para trabajar.

Ejemplo:

```
def saludar(nombre):
    print("Hola", nombre)
```

Aquí `nombre` es un parámetro.

Ahora podemos llamar la función pasando un valor:

```
saludar("Ana")
saludar("Luis")
```

Salida:

Hola Ana
Hola Luis

La función usa el valor recibido para completar su tarea.

---

7. Diferencia entre parámetro y argumento

Aunque muchas veces se usan como sinónimos, existe una diferencia sencilla:

Concepto	Explicación	Ejemplo
Parámetro	Variable que recibe la función	`nombre`
Argumento	Valor que se envía a la función	`"Ana"`

Ejemplo:

```
def saludar(nombre):
    print("Hola", nombre)

saludar("Ana")
```

En este caso:

nombre

es el parámetro.

"Ana"

es el argumento.

---

8. Funciones con más de un parámetro

Una función puede recibir varios datos.

```
def sumar(numero1, numero2):
    resultado = numero1 + numero2
    print("El resultado es:", resultado)
```

Llamada de la función:

```
sumar(5, 3)
```

Salida:

El resultado es: 8

Otro ejemplo:

```
def mostrar_persona(nombre, edad):
    print("Nombre:", nombre)
    print("Edad:", edad)

mostrar_persona("Camila", 21)
```

Salida:

Nombre: Camila
Edad: 21

---

9. Funciones que retornan valores

A veces una función no solo debe mostrar información, sino también **devolver un resultado**.

Para eso usamos `return`.

```
def sumar(numero1, numero2):
    resultado = numero1 + numero2
    return resultado
```

Luego podemos guardar el resultado en una variable:

```
total = sumar(5, 3)

print(total)
```

Salida:

8

Programa completo:

```
def sumar(numero1, numero2):
    resultado = numero1 + numero2
    return resultado

total = sumar(5, 3)

print("El total es:", total)
```

Salida:

El total es: 8

---

10. Diferencia entre `print()` y `return`

Esta diferencia es muy importante.

`print()` muestra algo en pantalla.

`return` devuelve un valor para que pueda ser usado después.

Ejemplo con `print()`:

```
def sumar(numero1, numero2):
    resultado = numero1 + numero2
    print(resultado)

sumar(5, 3)
```

Salida:

8

En este caso, la función muestra el resultado, pero no lo entrega para usarlo después.

Ejemplo con `return`:

```
def sumar(numero1, numero2):
    resultado = numero1 + numero2
    return resultado

total = sumar(5, 3)
```

```
print(total * 2)
```

Salida:

16

Aquí sí podemos reutilizar el resultado.

---

11. Ejemplo simple con retorno

```
def calcular_doble(numero):
    doble = numero * 2
    return doble

resultado = calcular_doble(10)

print(resultado)
```

Salida:

20

También se puede escribir más corto:

```
def calcular_doble(numero):
    return numero * 2

resultado = calcular_doble(10)

print(resultado)
```

---

12. Función para calcular promedio

```
def calcular_promedio(nota1, nota2, nota3):
    promedio = (nota1 + nota2 + nota3) / 3
    return promedio

resultado = calcular_promedio(5.5, 6.0, 7.0)
```

```
print("El promedio es:", resultado)
```

Salida:

```
El promedio es: 6.166666666666667
```

Podemos mejorar la salida usando `round()`:

```
def calcular_promedio(nota1, nota2, nota3):
    promedio = (nota1 + nota2 + nota3) / 3
    return promedio

resultado = calcular_promedio(5.5, 6.0, 7.0)

print("El promedio es:", round(resultado, 1))
```

Salida:

```
El promedio es: 6.2
```

---

13. Funciones con listas

Una función también puede recibir una lista como parámetro.

```
def mostrar_nombres(nombres):
    for nombre in nombres:
        print(nombre)

lista_nombres = ["Ana", "Luis", "Camila"]

mostrar_nombres(lista_nombres)
```

Salida:

```
Ana
Luis
Camila
```

Otro ejemplo: calcular el promedio de una lista de notas.

```
def calcular_promedio(notas):
```

```
    promedio = sum(notas) / len(notas)
    return promedio

notas_estudiante = [5.5, 6.0, 4.8, 7.0]

resultado = calcular_promedio(notas_estudiante)

print("Promedio:", resultado)
```

Salida:

```
Promedio: 5.825
```

---

14. Funciones con diccionarios

Una función puede recibir un diccionario.

```
def mostrar_estudiante(estudiante):
    print("Nombre:", estudiante["nombre"])
    print("Edad:", estudiante["edad"])
    print("Nota:", estudiante["nota"])

datos_estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}

mostrar_estudiante(datos_estudiante)
```

Salida:

```
Nombre: Ana
Edad: 20
Nota: 6.5
```

También podemos trabajar con una lista de diccionarios:

```
def mostrar_estudiantes(estudiantes):
    for estudiante in estudiantes:
        print("Nombre:", estudiante["nombre"])
        print("Nota:", estudiante["nota"])
```

```
        print("-------------")

lista_estudiantes = [
    {"nombre": "Ana", "nota": 6.5},
    {"nombre": "Luis", "nota": 5.8},
    {"nombre": "Camila", "nota": 6.9}
]

mostrar_estudiantes(lista_estudiantes)
```

---

15. Funciones con datos ingresados por el usuario

Podemos usar funciones junto con `input()`.

```
def pedir_nombre():
    nombre = input("Ingrese su nombre: ")
    return nombre

def saludar(nombre):
    print("Hola", nombre)

nombre_usuario = pedir_nombre()

saludar(nombre_usuario)
```

Ejemplo de ejecución:

```
Ingrese su nombre: Ana
Hola Ana
```

En este programa hay dos funciones:

Función	Tarea
`pedir_nombre()`	Solicita un nombre al usuario
`saludar(nombre)`	Muestra un saludo usando ese nombre

---

16. Parámetros con valores por defecto

Una función puede tener parámetros con valores por defecto.

```
def saludar(nombre="estudiante"):
    print("Hola", nombre)

saludar("Ana")
saludar()
```

Salida:

```
Hola Ana
Hola estudiante
```

Cuando no enviamos un argumento, Python usa el valor por defecto.

Otro ejemplo:

```
def calcular_descuento(precio, descuento=0.10):
    total = precio - (precio * descuento)
    return total

print(calcular_descuento(10000))
print(calcular_descuento(10000, 0.20))
```

Salida:

```
9000.0
8000.0
```

En la primera llamada se aplica un descuento de 10%.

En la segunda llamada se aplica un descuento de 20%.

---

17. Argumentos por posición

Normalmente los argumentos se envían en el mismo orden de los parámetros.

```
def mostrar_datos(nombre, edad):
    print("Nombre:", nombre)
```

```
    print("Edad:", edad)

mostrar_datos("Ana", 20)
```

Salida:

```
Nombre: Ana
Edad: 20
```

Aquí `"Ana"` llega al parámetro `nombre`.

Y `20` llega al parámetro `edad`.

---

18. Argumentos por nombre

También podemos indicar directamente qué valor corresponde a cada parámetro.

```
def mostrar_datos(nombre, edad):
    print("Nombre:", nombre)
    print("Edad:", edad)

mostrar_datos(edad=20, nombre="Ana")
```

Salida:

```
Nombre: Ana
Edad: 20
```

Aunque cambiamos el orden, Python entiende qué valor va en cada parámetro.

Esto es útil cuando una función tiene muchos parámetros.

---

19. Variables locales

Una variable creada dentro de una función se llama **variable local**.

Solo existe dentro de la función.

```
def mostrar_mensaje():
    mensaje = "Hola desde la función"
    print(mensaje)

mostrar_mensaje()
```

Salida:

```
Hola desde la función
```

Pero si intentamos usar la variable fuera de la función:

```
def mostrar_mensaje():
    mensaje = "Hola desde la función"

mostrar_mensaje()

print(mensaje)
```

Esto genera error porque `mensaje` solo existe dentro de la función.

---

20. Variables globales

Una variable creada fuera de una función se llama **variable global**.

```
mensaje = "Hola desde fuera"

def mostrar_mensaje():
    print(mensaje)

mostrar_mensaje()
```

Salida:

```
Hola desde fuera
```

La función puede leer una variable global.

Sin embargo, para estudiantes que están comenzando, es recomendable usar parámetros y retornos en vez de depender demasiado de variables globales.

Mejor opción:

```
def mostrar_mensaje(mensaje):
    print(mensaje)

texto = "Hola desde fuera"

mostrar_mensaje(texto)
```

---

21. Ejemplo práctico: calculadora simple

```
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b

def multiplicar(a, b):
    return a * b

def dividir(a, b):
    if b == 0:
        return "No se puede dividir por cero"
    else:
        return a / b

numero1 = float(input("Ingrese el primer número: "))
numero2 = float(input("Ingrese el segundo número: "))

print("Suma:", sumar(numero1, numero2))
print("Resta:", restar(numero1, numero2))
print("Multiplicación:", multiplicar(numero1, numero2))
print("División:", dividir(numero1, numero2))
```

Ejemplo de ejecución:

```
Ingrese el primer número: 10
Ingrese el segundo número: 5
Suma: 15.0
Resta: 5.0
Multiplicación: 50.0
División: 2.0
```

---

22. Ejemplo práctico: validar edad

```
def es_mayor_de_edad(edad):
    if edad >= 18:
        return True
    else:
        return False

edad_usuario = int(input("Ingrese su edad: "))

if es_mayor_de_edad(edad_usuario):
    print("Es mayor de edad")
else:
    print("Es menor de edad")
```

Ejemplo de ejecución:

```
Ingrese su edad: 20
Es mayor de edad
```

La función devuelve `True` o `False`.

Después usamos ese resultado en una condición `if`.

---

23. Ejemplo práctico: registrar estudiantes

```
def crear_estudiante():
    estudiante = {}

    estudiante["nombre"] = input("Ingrese nombre: ")
    estudiante["edad"] = int(input("Ingrese edad: "))
    estudiante["nota"] = float(input("Ingrese nota: "))

    return estudiante

def mostrar_estudiante(estudiante):
    print("Nombre:", estudiante["nombre"])
    print("Edad:", estudiante["edad"])
    print("Nota:", estudiante["nota"])
```

```
estudiantes = []

for i in range(3):
    nuevo_estudiante = crear_estudiante()
    estudiantes.append(nuevo_estudiante)

print("Estudiantes registrados:")

for estudiante in estudiantes:
    mostrar_estudiante(estudiante)
    print("-------------")
```

Este programa combina:

- Funciones.
- Diccionarios.
- Listas.
- Ciclos.
- Entrada de datos.
- Salida de datos.

---

24. Ejemplo práctico: menú con funciones

```
def mostrar_menu():
    print("1. Saludar")
    print("2. Sumar dos números")
    print("3. Salir")

def saludar():
    nombre = input("Ingrese su nombre: ")
    print("Hola", nombre)

def sumar_numeros():
    numero1 = float(input("Ingrese el primer número: "))
    numero2 = float(input("Ingrese el segundo número: "))
    resultado = numero1 + numero2
    print("Resultado:", resultado)

while True:
    mostrar_menu()

    opcion = input("Seleccione una opción: ")
```

```
    if opcion == "1":
        saludar()

    elif opcion == "2":
        sumar_numeros()

    elif opcion == "3":
        print("Programa finalizado")
        break

    else:
        print("Opción no válida")
```

Este tipo de estructura es muy útil para programas más grandes.

Cada opción del menú llama a una función diferente.

---

25. Funciones que llaman a otras funciones

Una función puede llamar a otra función.

```
def pedir_numero():
    numero = float(input("Ingrese un número: "))
    return numero

def sumar(a, b):
    return a + b

def ejecutar_suma():
    numero1 = pedir_numero()
    numero2 = pedir_numero()

    resultado = sumar(numero1, numero2)

    print("Resultado:", resultado)

ejecutar_suma()
```

Aquí ocurre lo siguiente:

1. `ejecutar_suma()` inicia el proceso.
2. Dentro de ella se llama a `pedir_numero()` dos veces.
3. Luego se llama a `sumar(a, b)`.
4. Finalmente se muestra el resultado.

---

26. Buenas prácticas al crear funciones

Usar nombres claros

Mal ejemplo:

```
def x(a, b):
    return a + b
```

Mejor ejemplo:

```
def sumar_numeros(numero1, numero2):
    return numero1 + numero2
```

El nombre de la función debe indicar qué hace.

Que una función haga una tarea específica

Mal ejemplo:

```
def hacer_todo():
    # Pide datos
    # Calcula promedio
    # Muestra menú
    # Guarda estudiantes
    pass
```

Mejor idea:

```
def pedir_datos():
    pass

def calcular_promedio():
    pass
```

```
def mostrar_menu():
    pass
```

Es mejor tener varias funciones pequeñas que una función enorme.

Evitar repetir código

Si una instrucción se repite muchas veces, probablemente conviene crear una función.

Ejemplo:

```
def mostrar_separador():
    print("--------------------")

mostrar_separador()
print("Estudiante: Ana")
mostrar_separador()
print("Estudiante: Luis")
mostrar_separador()
```

---

27. Errores comunes al trabajar con funciones

Error 1: Olvidar los paréntesis al llamar la función

Incorrecto:

```
def saludar():
    print("Hola")

saludar
```

Correcto:

```
saludar()
```

Sin paréntesis, la función no se ejecuta.

Error 2: Llamar una función antes de definirla

Incorrecto:

```
saludar()

def saludar():
    print("Hola")
```

Correcto:

```
def saludar():
    print("Hola")

saludar()
```

Python debe conocer la función antes de ejecutarla.

Error 3: Enviar menos argumentos de los necesarios

```
def sumar(a, b):
    return a + b

print(sumar(5))
```

Esto genera error porque la función espera dos argumentos.

Correcto:

```
print(sumar(5, 3))
```

Error 4: Confundir `print()` con `return`

```
def sumar(a, b):
    print(a + b)

resultado = sumar(5, 3)

print(resultado)
```

Salida:

8
None

¿Por qué aparece `None`?

Porque la función muestra el resultado con `print()`, pero no retorna ningún valor.

Correcto:

```
def sumar(a, b):
    return a + b

resultado = sumar(5, 3)

print(resultado)
```

Error 5: Olvidar la indentación

Incorrecto:

```
def saludar():
print("Hola")
```

Correcto:

```
def saludar():
    print("Hola")
```

---

28. Ejercicios propuestos

Ejercicio 1: Saludo simple

Crea una función llamada `saludar` que muestre el mensaje:

Hola, estoy aprendiendo funciones

Luego llama la función tres veces.

Ejercicio 2: Saludo personalizado

Crea una función llamada `saludar_persona` que reciba un nombre como parámetro y muestre:

Hola, [nombre]

Ejercicio 3: Sumar dos números

Crea una función llamada `sumar` que reciba dos números y retorne la suma.

Luego muestra el resultado por pantalla.

Ejercicio 4: Calcular promedio

Crea una función llamada `calcular_promedio` que reciba tres notas y retorne el promedio.

Ejercicio 5: Número par o impar

Crea una función llamada `es_par` que reciba un número.

La función debe retornar `True` si el número es par y `False` si es impar.

Ejercicio 6: Mayor de edad

Crea una función llamada `es_mayor_de_edad` que reciba una edad.

Debe retornar `True` si la edad es mayor o igual a 18, y `False` en caso contrario.

Ejercicio 7: Área de un rectángulo

Crea una función llamada `calcular_area_rectangulo` que reciba base y altura.

Debe retornar el área.

Fórmula:

área = base * altura

Ejercicio 8: Mostrar lista

Crea una función llamada `mostrar_lista` que reciba una lista y muestre todos sus elementos usando un ciclo `for`.

Ejercicio 9: Promedio de una lista

Crea una función llamada `promedio_lista` que reciba una lista de números.

Debe retornar el promedio de los números.

Ejercicio 10: Buscar estudiante

Crea una función que reciba una lista de estudiantes.

Cada estudiante debe ser un diccionario con:

- `nombre`
- `edad`
- `nota`

La función debe pedir un nombre y mostrar los datos del estudiante si existe.

---

29. Resumen de conceptos importantes

Concepto	Explicación	Ejemplo

`def`	Permite crear una función	`def saludar():`
Parámetro	Variable que recibe la función	`def saludar(nombre):`
Argumento	Valor enviado a la función	`saludar("Ana")`
`return`	Devuelve un resultado	`return resultado`
`print()`	Muestra información en pantalla	`print("Hola")`
Variable local	Existe dentro de la función	`resultado = a + b`
Variable global	Existe fuera de la función	`mensaje = "Hola"`
Valor por defecto	Parámetro con valor inicial	`def saludar(nombre="estudiante"):`

---

30. Resumen de estructura básica

Función sin parámetros:

```
def saludar():
    print("Hola")

saludar()
```

Función con parámetros:

```
def saludar(nombre):
    print("Hola", nombre)

saludar("Ana")
```

Función con retorno:

```
def sumar(a, b):
    return a + b

resultado = sumar(5, 3)

print(resultado)
```

Función con lista:

```
def mostrar_numeros(numeros):
    for numero in numeros:
        print(numero)

mostrar_numeros([1, 2, 3, 4])
```

Función con diccionario:

```
def mostrar_persona(persona):
    print(persona["nombre"])
    print(persona["edad"])

mostrar_persona({"nombre": "Ana", "edad": 20})
```

Cierre

Las funciones son una parte fundamental de Python y de la programación en general.

Permiten ordenar el código, reutilizar instrucciones y dividir un programa en partes más pequeñas.

Antes de usar funciones, normalmente escribimos todo el código junto. Pero cuando los programas crecen, eso se vuelve difícil de entender.

Por eso, aprender funciones es un paso clave para avanzar hacia temas como:

- Programación orientada a objetos.
- Creación de menús.
- Desarrollo web.
- Automatización.
- Pruebas unitarias.
- APIs.
- Proyectos más grandes en Python.